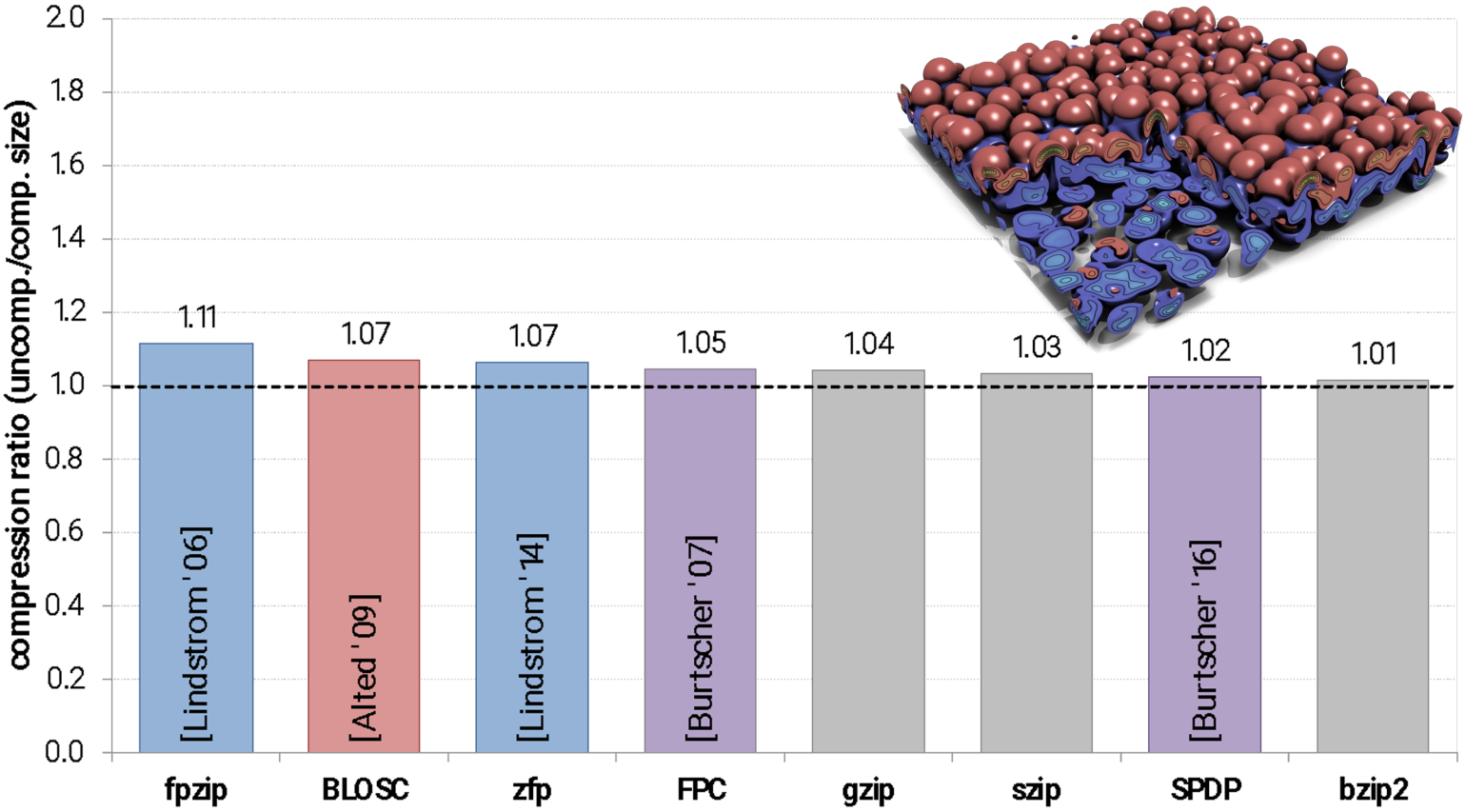


Section 2: Compression Techniques

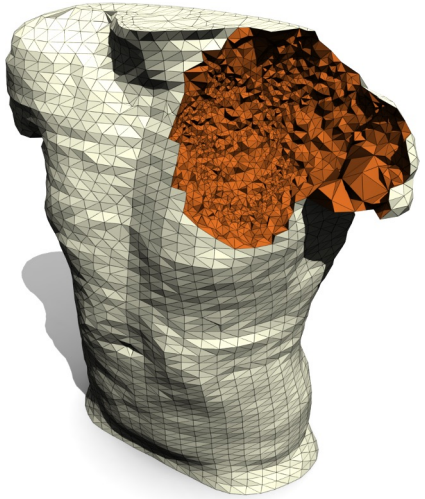
Peter Lindstrom, Lawrence Livermore National Laboratory

Numerical data is challenging to compress losslessly



Numerical data is challenging to compress for several reasons

- High precision—usually 32 to 64 bits
  - Unlike in text, few recurring values (if any!) in a data set
  - Many (most?) least significant bits are “random”…
    - … and represent sensor, round-off, truncation, iteration error



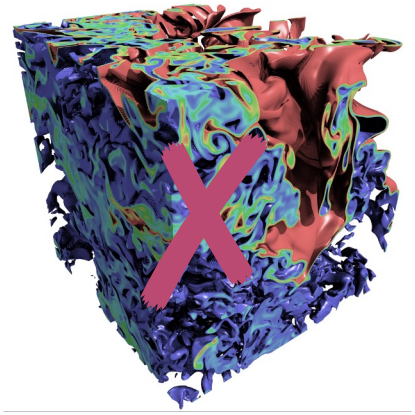
- Data may be unstructured/unordered
  - E.g., unrelated particles and fields, unstructured mesh
  - No natural ordering of many-field data

- But leading bits tend to be predictable
  - Aside from physical boundaries, shocks, etc., fields tend to be (at least) C^0 continuous
  - For numerical stability, grid resolution tends to be high enough to ensure smoothness
  - Fields are sometimes correlated (cf. ideal gas law)

- Lossy compression **approximates** data well using far less storage

Disclaimers

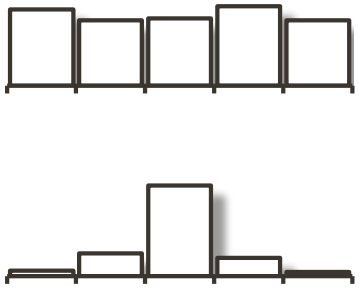
- This talk primarily focuses on compression of…
  - Floating-point scalars (single and double precision)
  - Structured data defined on (logically) Cartesian grids (1D, 2D, 3D, 3D + time)
    - Generalizations to unstructured data exist, but are beyond the scope of this tutorial
  - Data that is correlated along each dimension
    - E.g., continuous (ideally smooth) discretely sampled functions like PDE solutions
    - Uncorrelated dimensions treated separately (e.g., velocity components)
- Although emphasis is on scientific data, natural images will be used for illustration
  - Easier for human eye to discern shape and quality in 2D photographs than in 3D “blobs”





Numerical data compression usually involves three steps

- 1. **Decorrelate** the data to make it more compressible
  - E.g., prediction, fitting, transformations, decompositions, …
  - Goal is to make the data sparse in some alternative representation
  - Small values and repeated patterns are easier to compress

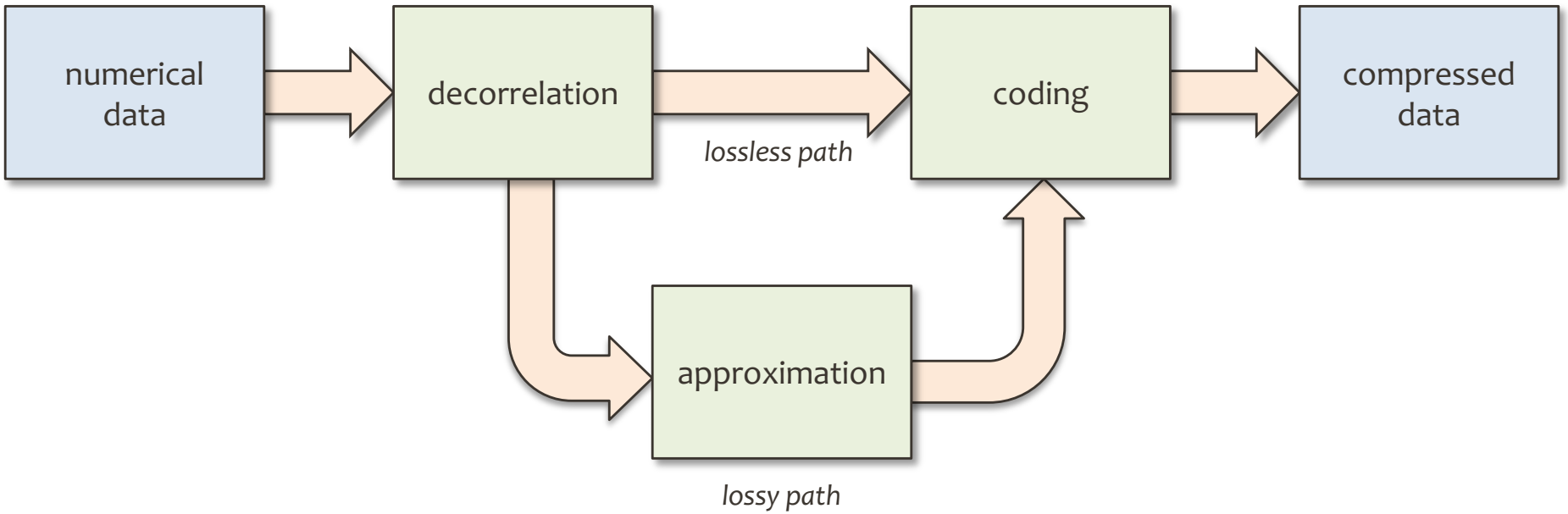


- 2. **Approximate** (for lossy compression)
  - E.g., scalar/vector quantization, truncation, thresholding, filtering, …
  - Discard unimportant information, thus avoiding having to encode it

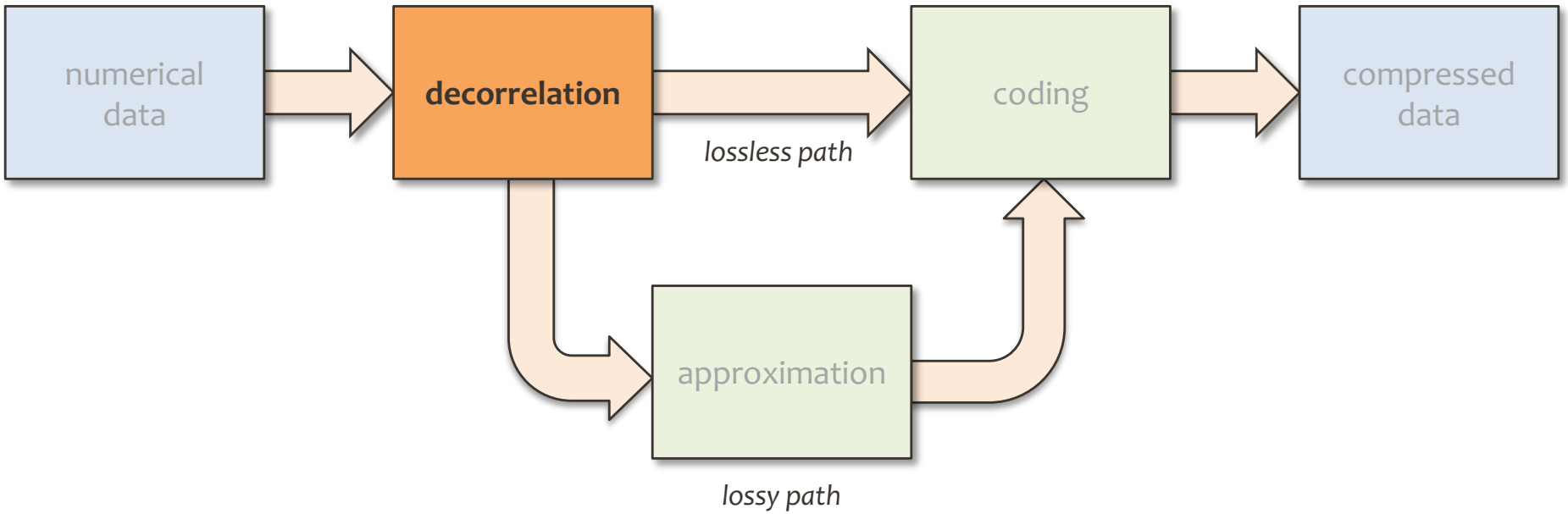
- 3. **Encode** remaining information losslessly and/or progressively
  - E.g., Huffman, arithmetic, universal, run-length, dictionary, embedded, …



Numerical compression pipeline



Numerical compression pipeline

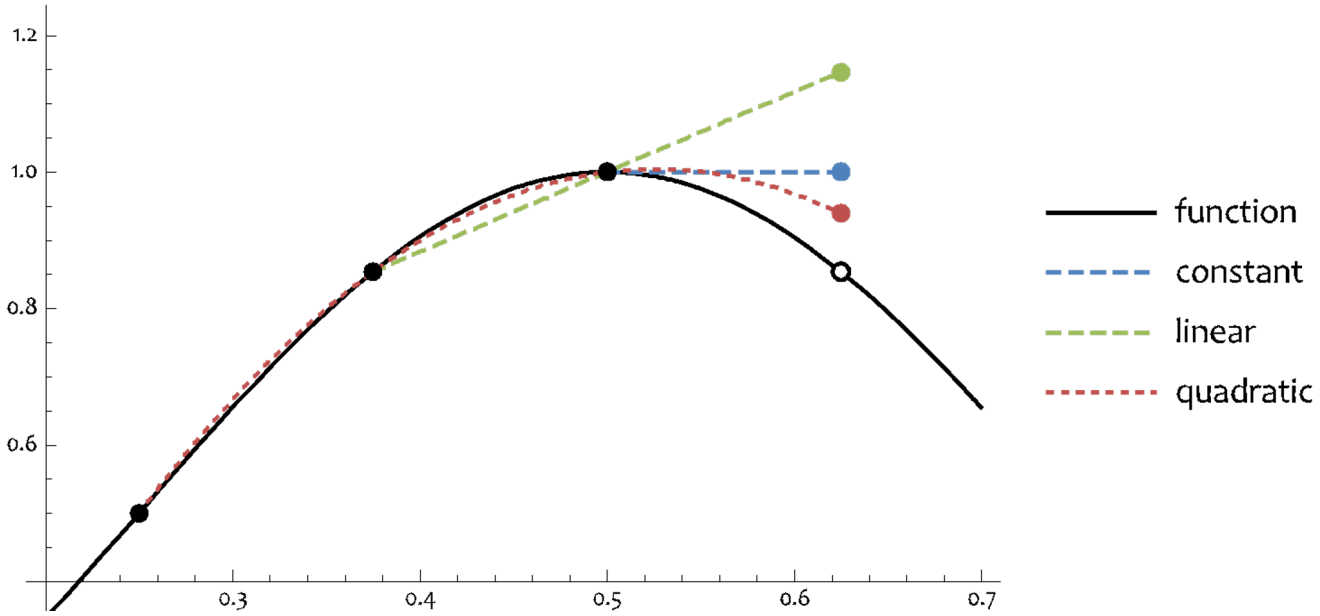


Decorrelation: Prediction/correction

- Idea: Generate prediction, p, of next value, f, from recent history of values
  - Usually as weighted average: $p_i = w_{i-1} f_{i-1} + w_{i-2} f_{i-2} + \cdots \approx f_i$
- Compute residual $r = f - p$
  - Residuals tend to be much smaller and have lower entropy than actual values
- Optionally quantize residual
- Compressor encodes and transmits residual
- Decompressor recovers $f = p + r$ from same prediction, p, and decoded residual, r

Decorrelation: Polynomial prediction

- Polynomial of degree *n* – 1 predicts next value from last *n* transmitted values
  - Used in older versions of [SZ] compressor



Decorrelation: Lorenzo prediction [LP]

- 1D polynomial predictor interpolates *n* points and everywhere satisfies

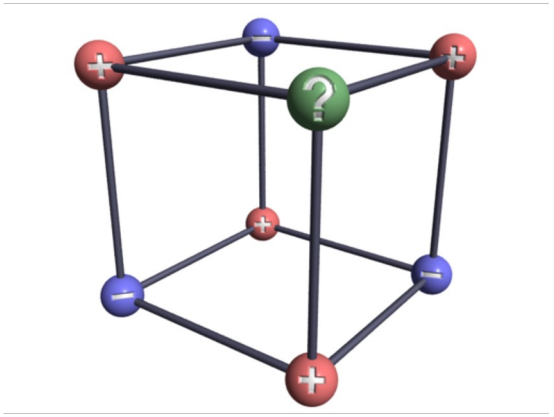
$$\frac{\partial^n f}{\partial x^n} = 0$$

- For *d*-dimensional data on structured grid, **Lorenzo predictor** satisfies

$$\frac{\partial^d f}{\partial x_1 \partial x_2 \cdots \partial x_d} = 0$$

- Lorenzo predictor is given by sums and differences of $2^d - 1$ values

- Used in [FPZIP], [SZ] compressors

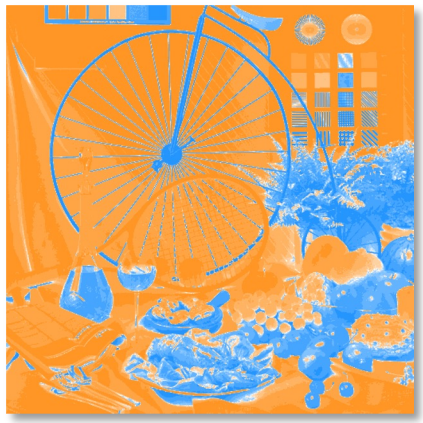


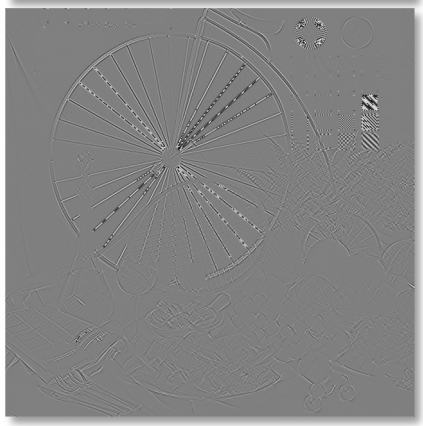
Decorrelation: Lorenzo prediction example

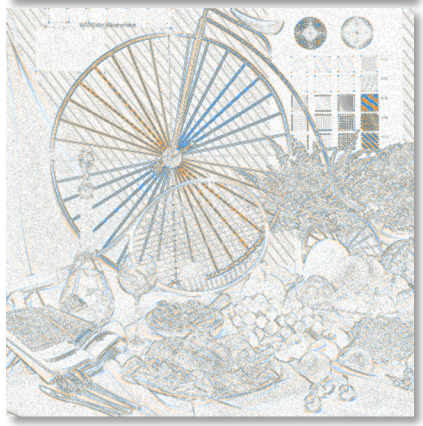
actual

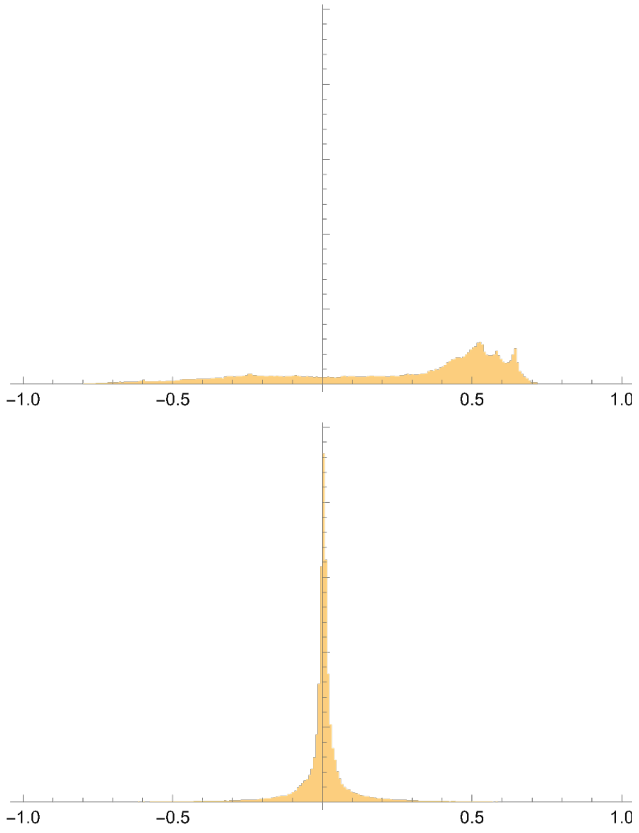
residual











histograms

Decorrelation: Linear transforms

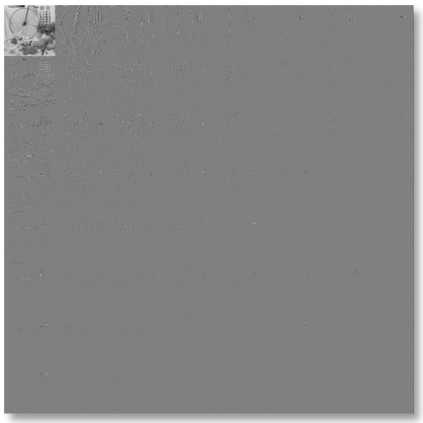
- Project data, x, onto a basis, A, in which it is sparser: $y = A\,x$
- Ideally the transform is orthogonal
  - Minimizes coefficient magnitudes
  - Simple inverse: matrix transpose
- Many, many transforms to choose from
  - Spectral (Fourier, discrete cosine, discrete sine, Walsh-Hadamard)
  - Wavelets (Haar, Daubechies, B-spline, CDF, …)
  - Gram polynomials (aka. discrete Legendre transform)
  - Slant
  - Karhunen-Loève
  - …
- NOTE: Predictor can also be viewed as lower triangular linear transform
  - $A = I - P$, with strictly lower triangular prediction matrix P

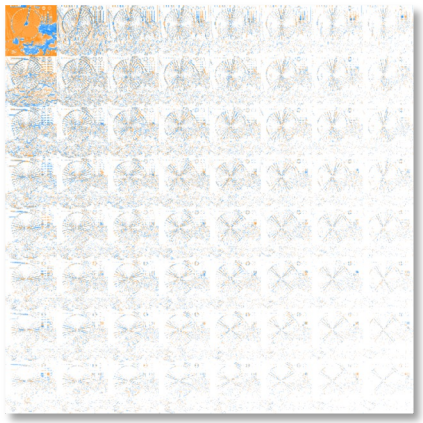
Decorrelation: Discrete Cosine Transform (DCT)

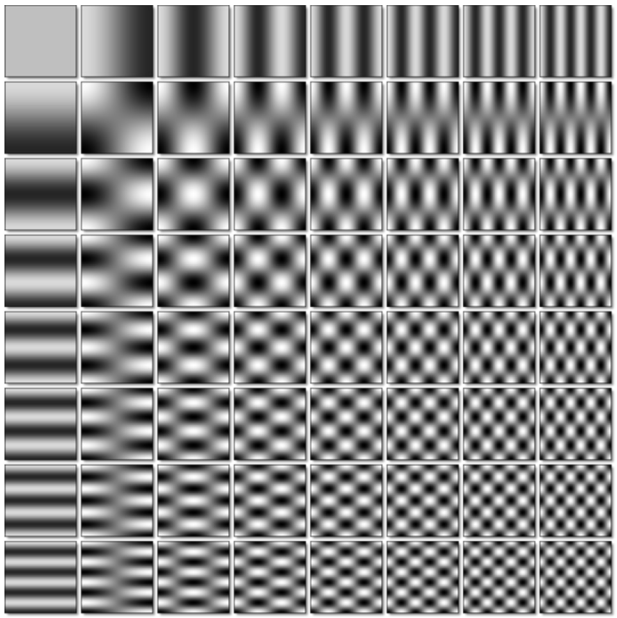
- Real-valued transform with sinusoidal basis functions

$$x_{ij} = \cos\left(\frac{\pi}{n}\left(i + \tfrac{1}{2}\right)j\right) \qquad 0 \le i, j \le n - 1$$

- Multidimensional transform formed by tensor product
  - Apply transform independently along x, then y, then z, …







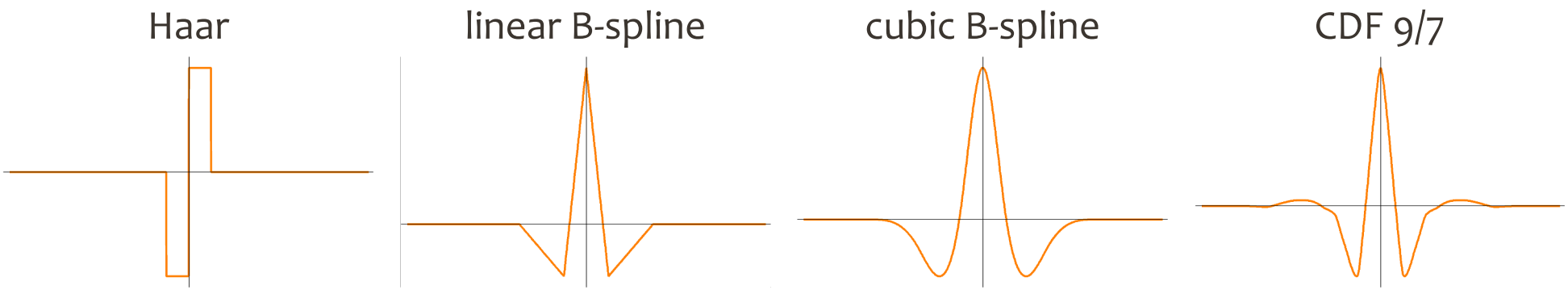
2D DCT basis functions

Decorrelation: Discrete Wavelet Transform (DWT)

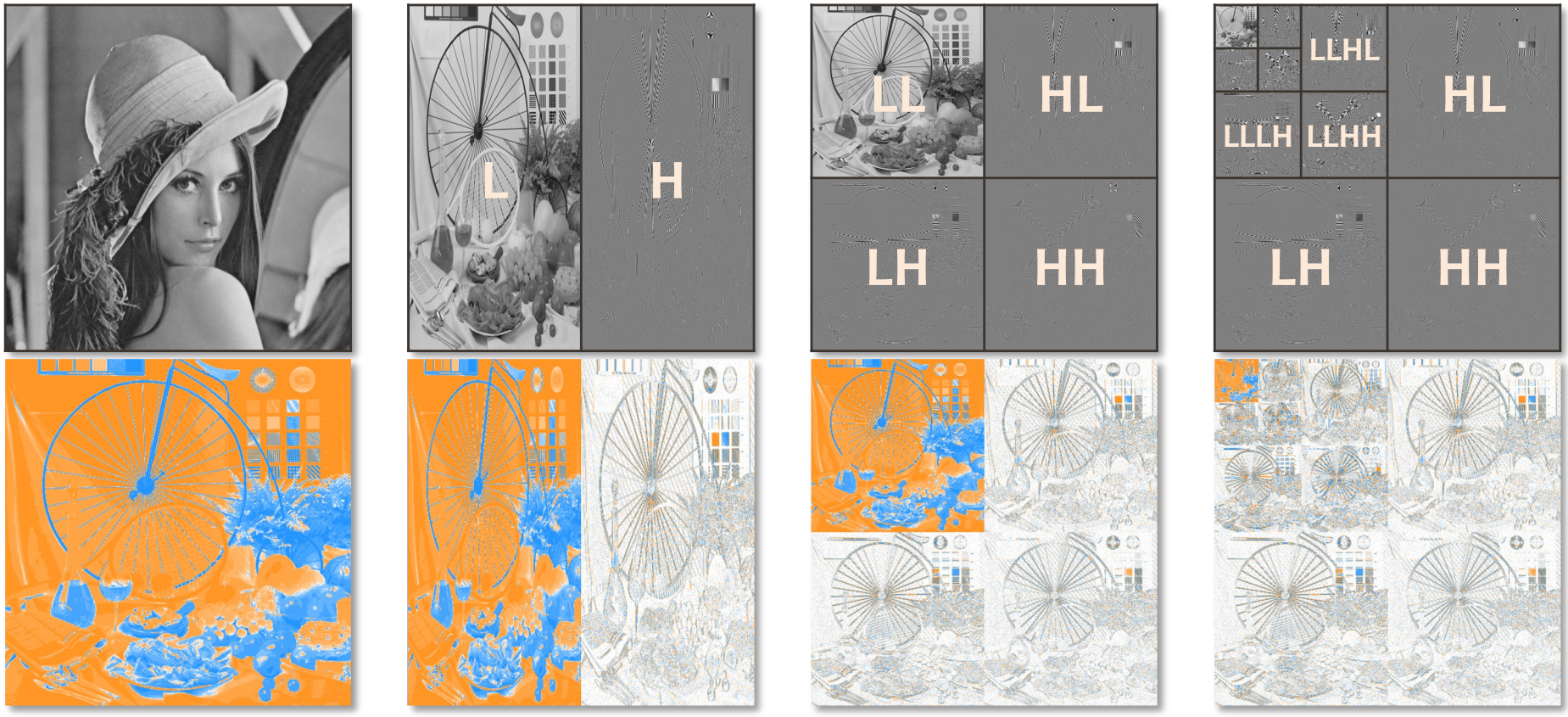
- Basis functions given by translations and dilations of single mother wavelet ψ

$$\psi_{i,\ell}(x) = 2^{-\ell/2}\psi\left(\frac{x-i}{2^\ell}\right)$$

- Common in image and volume data compression
  - [JPEG2000], [MGARD], [SPERR], [VAPOR], many others



Decorrelation: Discrete Wavelet Transform (DWT)



Decorrelation: Singular Value Decomposition (SVD)

- 2D structured grid data, *A*, can be approximated via truncated SVD

$$\mathrm{diag}(\sigma_1, \sigma_2, \ldots, \sigma_n) = \Sigma = U^T A V = \mathrm{unvec}\big((V \otimes U)^T \mathrm{vec}(A)\big)$$

  - Singular value matrix, Σ, is **diagonal** but singular vectors, *U* and *V*, are **data-dependent**
  - *A* can be optimally approximated in the L_2 sense by discarding smallest singular values

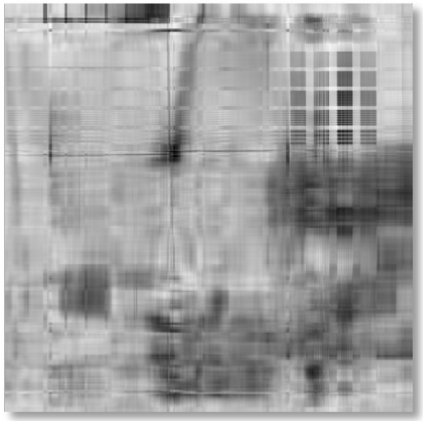
- Contrast with separable (tensor product) 2D wavelet transform

$$C = W^T A W = \mathrm{unvec}\big((W \otimes W)^T \mathrm{vec}(A)\big)$$

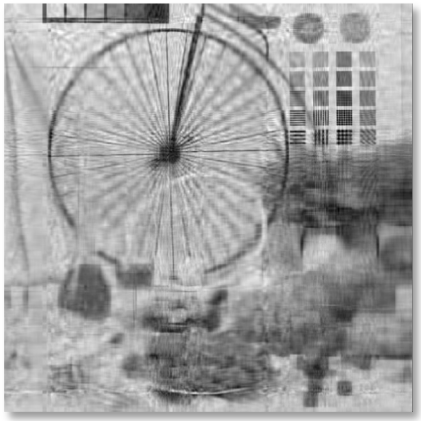
  - Wavelet coefficient matrix, *C*, is **dense** but basis, *W,* is **data-independent**

Decorrelation: Singular Value Decomposition (SVD)

- Data-dependent “optimal” basis must be encoded explicitly
  - Orthonormality constraints on *U*, *V* reduce number of degrees of freedom
  - Best rank-*r* approximation of *n*-by-*n* matrix requires $r \times (2n - r)$ coefficients



3% of coefficients



10% of coefficients

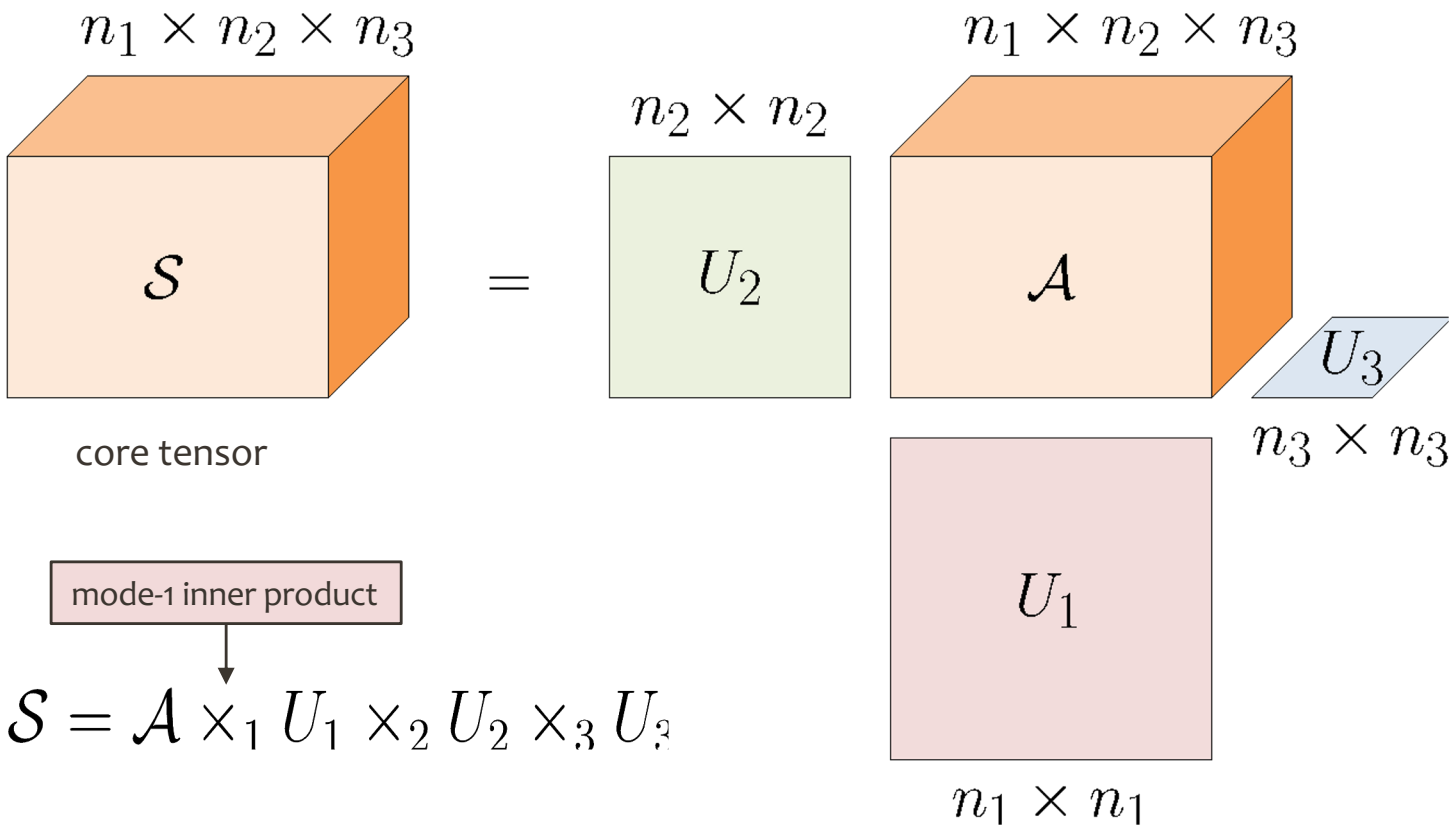


30% of coefficients

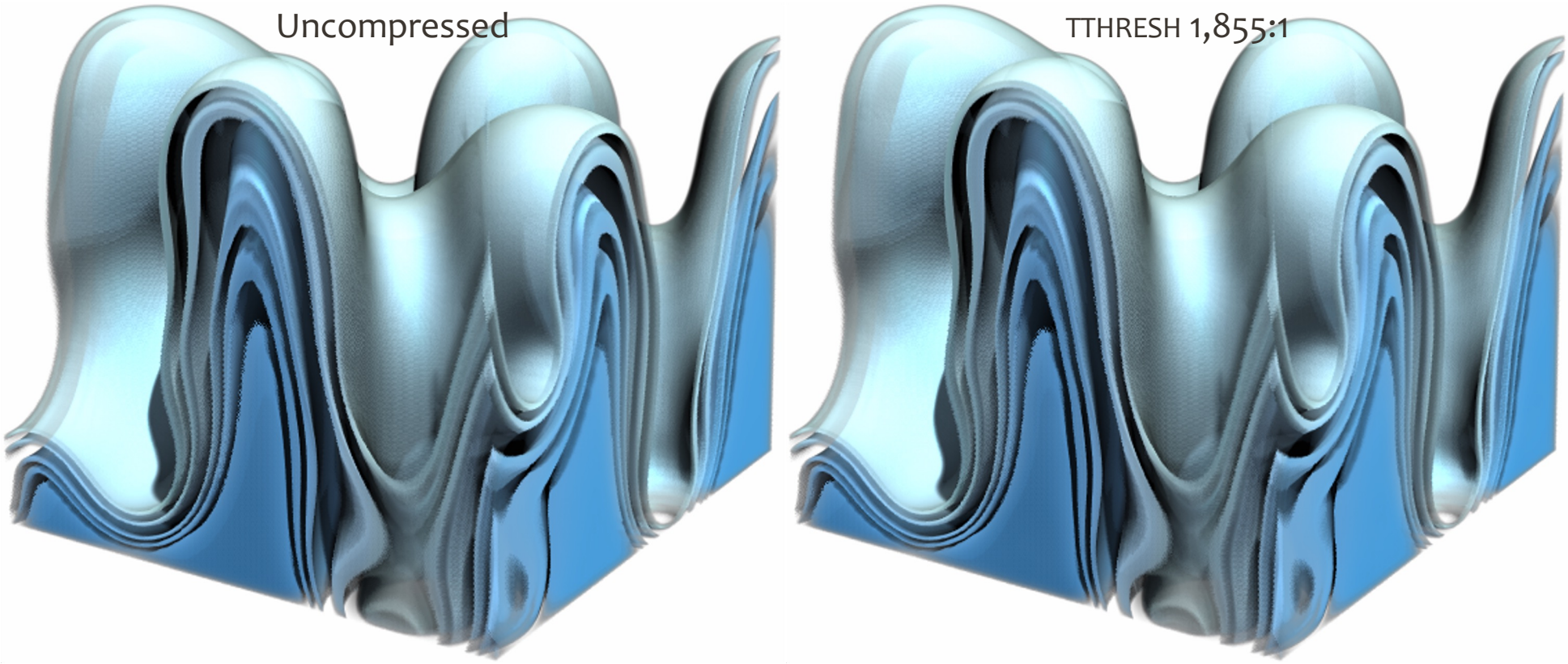
Decorrelation: Tensor decomposition

- SVD is rarely used for 2D data compression due to high cost of coding basis
  - The situation is quite different for higher dimensional data
- For *d*-dimensional structured grids ($d \geq 3$), use multilinear tensor decomposition
  - Generalization of SVD to higher dimensions
- Several tensor decompositions exist
  - **CP**: Weighted sum of outer products of (non-orthogonal) basis vectors
  - **Tucker**: Unlike in 2D, "core tensor" *S* is dense (not diagonal)
  - **Tensor train**: Most suitable for higher dimensional data ($d \geq 4$)
- Data-dependent basis is relatively cheap to encode in 3D
  - Orthogonal 2D factor matrices are very small in relation to 3D data
  - As in SVD, we may truncate factor matrices and core tensor

Decorrelation: [TTHRESH] tensor decomposition

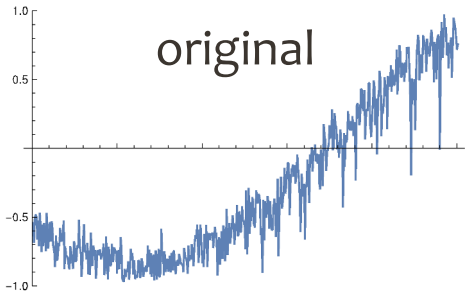


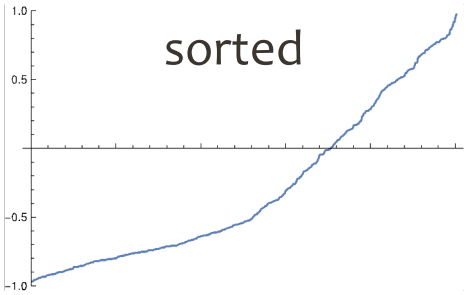
TTHRESH performs very well at high compression ratios

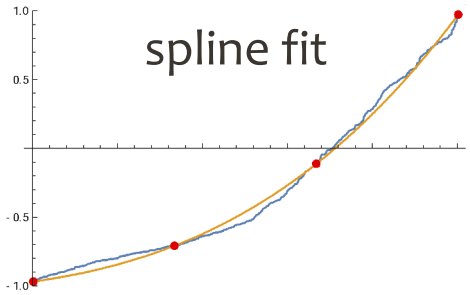


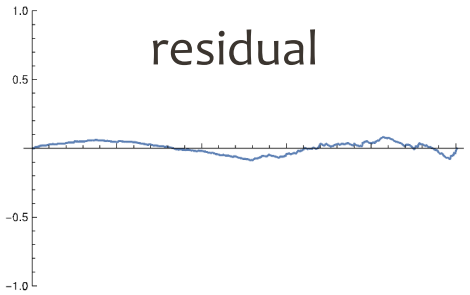
Decorrelation: Sorting and fitting

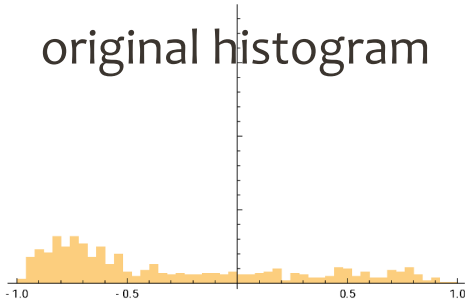
- Most compression techniques fail miserably on noisy/unstructured data
- [ISABELA]: Sort noisy data, encode permutation, fit “smooth” sorted signal

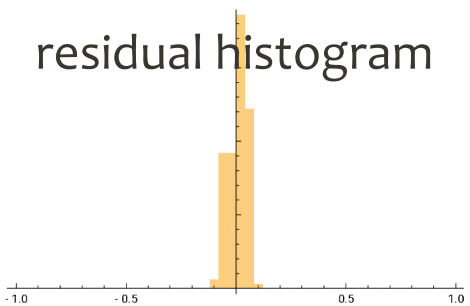




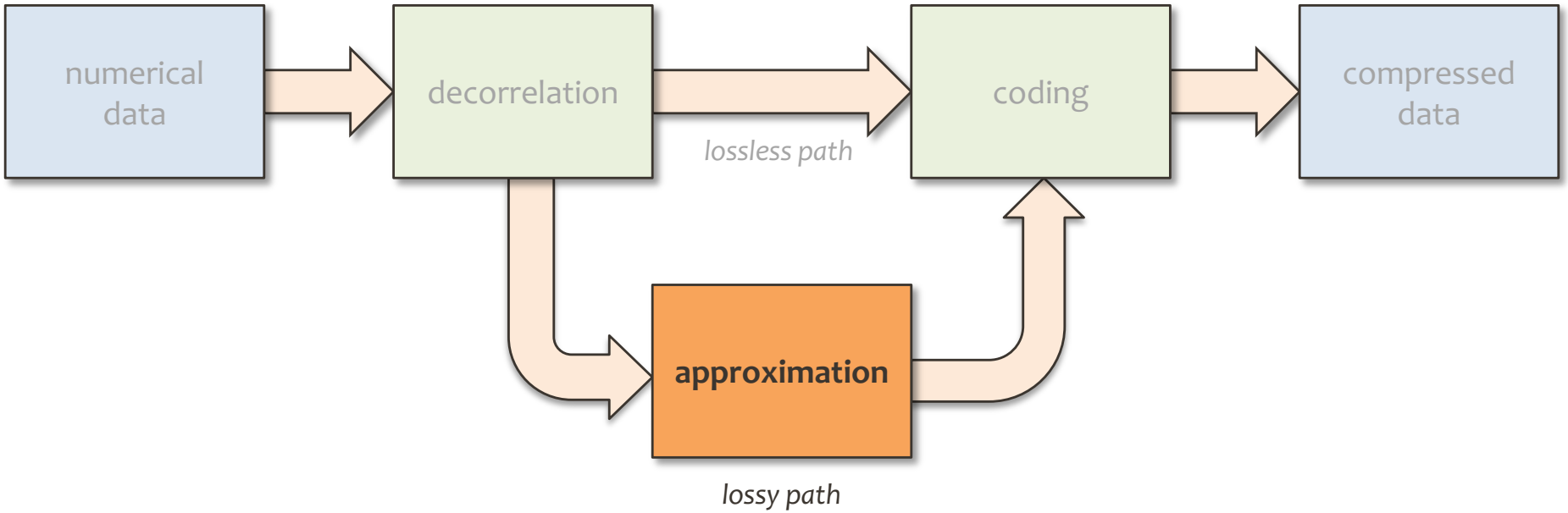






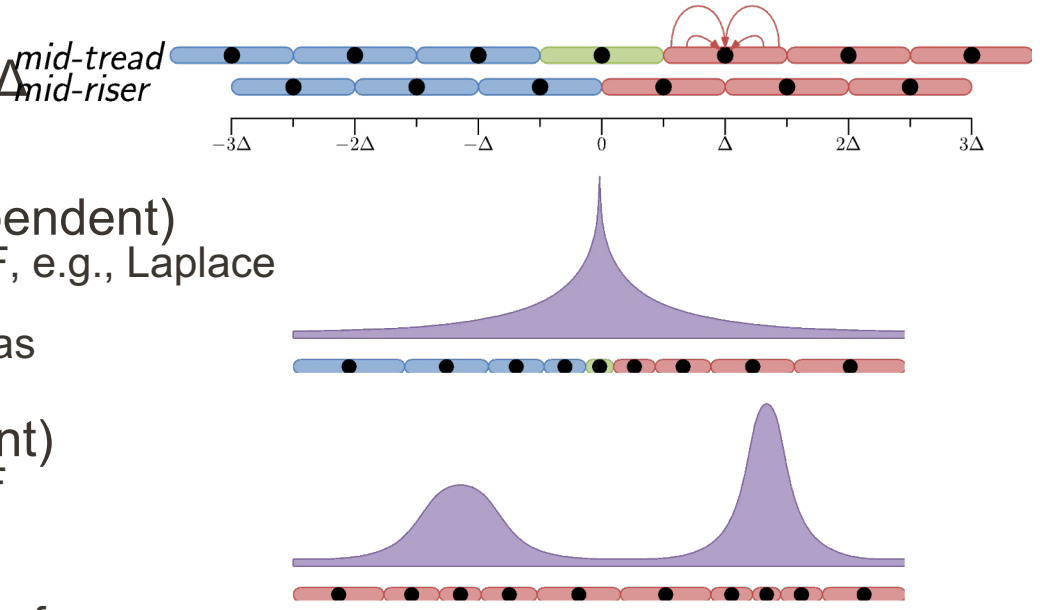


Numerical compression pipeline



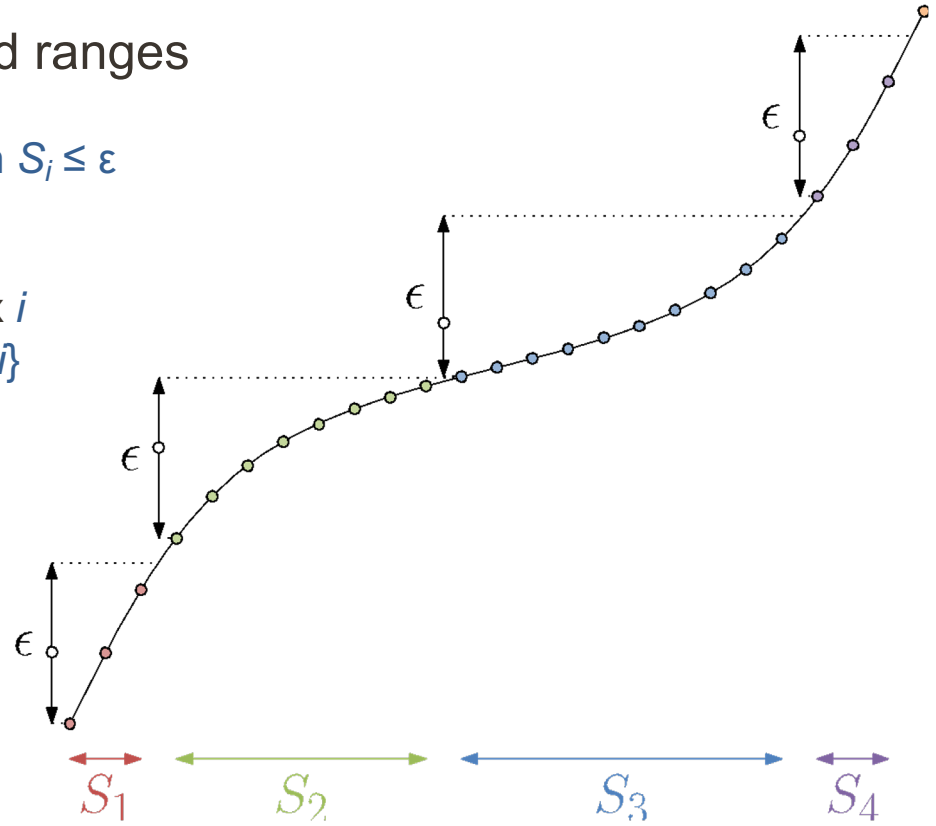
Approximation: Scalar quantization

- Scalar quantization: replace real value with index into codebook
  - Codebook: discrete set of representable values (aka. prototypes)
  - May be given implicitly by formula
- Uniform in range: fixed step size Δ
  - Mid-tread vs. mid-riser
- Non-uniform in range (data-independent)
  - Adapt quantization to assumed PDF, e.g., Laplace
  - Logarithmic quantization
  - Truncation of floating-point mantissas
- Adaptive in range (data-dependent)
  - Adapt quantization to empirical PDF
- Non-uniform in domain
  - Adapt quantization levels in space or frequency
  - JPEG: coarse quantization of high-frequency coefficients



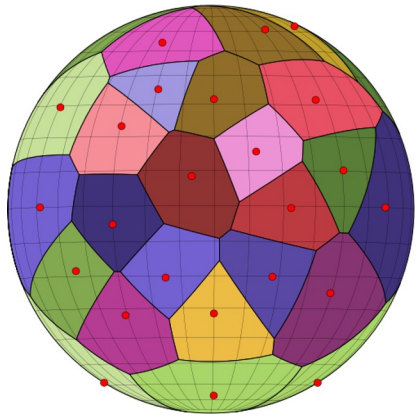
Approximation: Adaptive, error-bounded scalar quantization

- [SQ] algorithm partitions data into ε-sized ranges
  - Sort data on function value
  - Greedily grow set S_i as long as max S_i – min $S_i \leq \varepsilon$
  - Use as prototype p_i = mean S_i
    - Minimizes RMS error
  - Replace values assigned to set S_i with index i
  - LZMA compress codebook $\{p_i\}$ and indices $\{i\}$



Approximation: Vector Quantization (VQ)

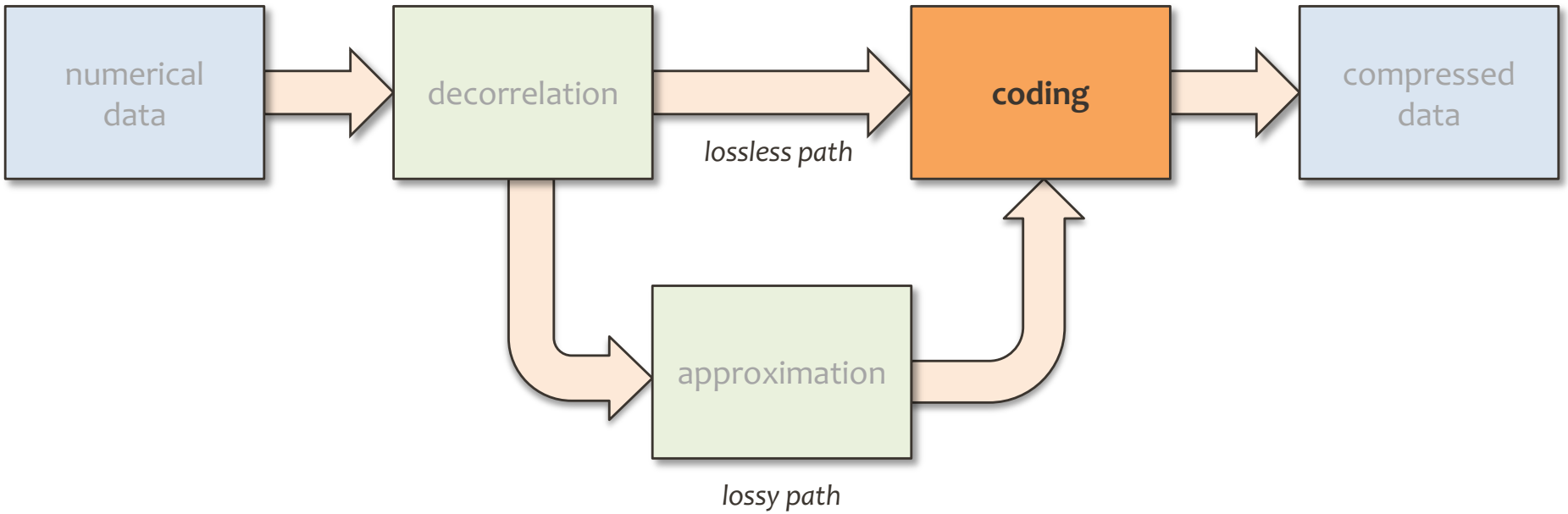
- Similar to scalar quantization, but applied to multi-component vectors
  - E.g., vector and tensor fields, multiple correlated fields, blocks of values, …
- Can be done non-uniformly in both domain and range
  - E.g., hierarchical VQ uses different codebook on each level [HVQ]
- Codebook is usually generated using Lloyd relaxation
  - Randomly select initial prototypes
  - Partition data by closest prototype
  - Replace prototype with mean/medoid/Voronoi centroid
- Most effective for low-precision data like 8-bit RGB
  - Codebook size and compute time become prohibitive for higher precision



Approximation: Thresholding, truncation, & filtering

- Decorrelation usually results in many small values that can be filtered out
  - If combined with quantization, values quantized to zero may be excluded
- **Thresholding**
  - Exclude all coefficients/residuals smaller than a given threshold
  - **Significance map** locates values above threshold
- **Truncation**
  - Keep prescribed number of largest coefficients
  - Also requires significance map unless coefficients are already sorted, as in SVD
- **Filtering**
  - Exclusion by sub-band, e.g., exclude highest frequency modes

Numerical compression pipeline



Coding: Assignment of numerical values to bit strings

- At this point, we're left with integer, fixed-point, or floating-point values
  - Some values may already have been discarded in thresholding, truncation, filtering
- Coding is where compression actually happens
  - In the simplest case, each value is mapped to a **codeword** (bit string)
  - We usually use shorter codewords for smaller and more frequent coefficients/residuals
  - Coding is (often) **lossless**

$$\mathbb{R} \Longrightarrow 1011010001\ldots$$

Numerical coding maps reals, $\mathbb{R}$, to bit strings, $\mathbb{B}^n$, in stages

1. Floating-point data is first reduced to signed integer data: $\mathbb{R} \Longrightarrow \mathbb{Z}$
   - **Quantization**: round float to integer
   - **Reinterpretation**: treat binary float as sign-magnitude integer
   - **Block-floating-point**: block of integers scaled by common exponent

2. Signed integers are then converted to unsigned: $\mathbb{Z} \Longrightarrow \mathbb{N}$
   - Reversible mapping that orders signed values roughly by magnitude
     - E.g., sign-magnitude, zig-zag, negabinary, …

3. Unsigned integers are finally mapped to bit strings: $\mathbb{N} \Longrightarrow \mathbb{B}^n$
   - Compress away leading zeros
     - E.g., universal code, interpolative code, Huffman code, arithmetic code, …

Signed-to-unsigned integer mapping involves optimizing two sometimes conflicting goals

- **Goal 1**: Preserve magnitude $y \approx |x|$
  - Small signed values, x, near zero should map to small unsigned values, y (symbols)
    - That is, symbol has many **compressible leading zeros**
  - Bits to the right of the leading one-bit in y are **quasi-random and incompressible**
    - Good decorrelation produces small values = few random bits to code

- **Goal 2**: Be resilient to approximation/rounding/quantization/progressive coding
  - Minimize error associated with reduced precision
    - E.g., when truncating trailing bits

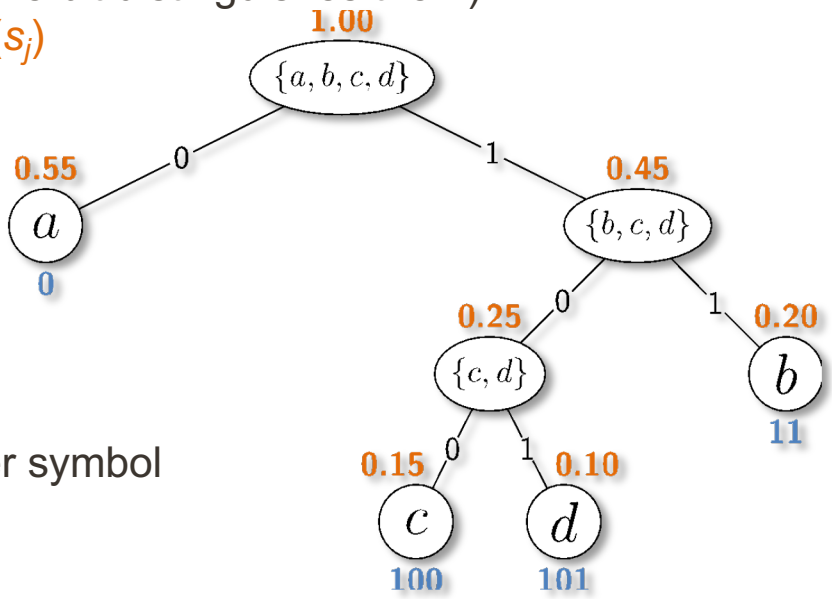
Choice of coding scheme depends on knowledge of data distribution

- Example coding schemes for numerical data ($\mathbb{N} \Longrightarrow \mathbb{B}^n$)
  - Assumed distribution: data-independent **universal codes** (Golomb, Fibonacci, Elias)
  - Empirical distribution: data-dependent **entropy coding** (Huffman, arithmetic)
  - Correlated residuals: **vector coding schemes** (RBUC, interpolative, tournament)
  - Runs of constant values: **run-length coding**

- Coding can also be applied to byte string representation, e.g., using gzip, bzip2, …
  - Most effective when coupled with **byte shuffling**
    - Transpose bytes so all MSBs appear first, LSBs appear last

Scalar coding: Entropy codes

- **Huffman code**: optimal prefix-free code for whole-bit codewords
  - Code for finite alphabet given known (or estimated) symbol probabilities
  - Idea: Generate binary code tree bottom up in greedy manner
  - Make two least probable symbols s_i, s_j siblings (one bit distinguishes them)
  - Replace siblings with parent: $p(\{s_i, s_j\}) = p(s_i) + p(s_j)$
  - Recurse until a single root node remains
  - Code of symbol given by path from root to leaf

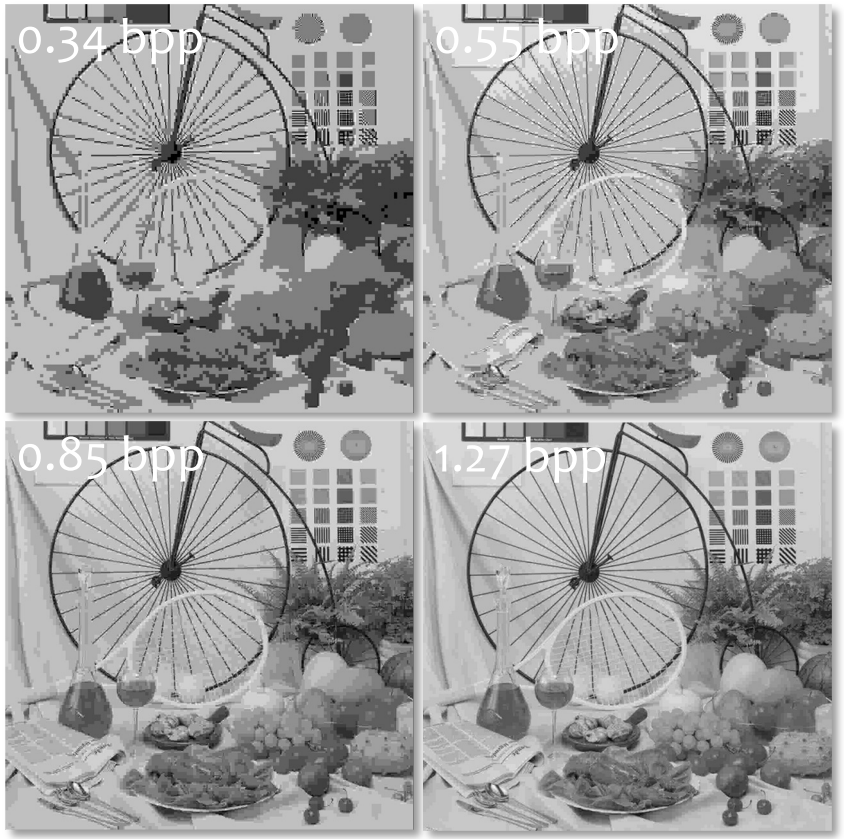


- **Shannon-Fano code**
  - Similar to Huffman, but built top-down

- **Arithmetic code**
  - Optimal code that uses exactly $-\log_2(p(s))$ bits per symbol
    - Approaches entropy with data size
  - Codewords generally have fractional width

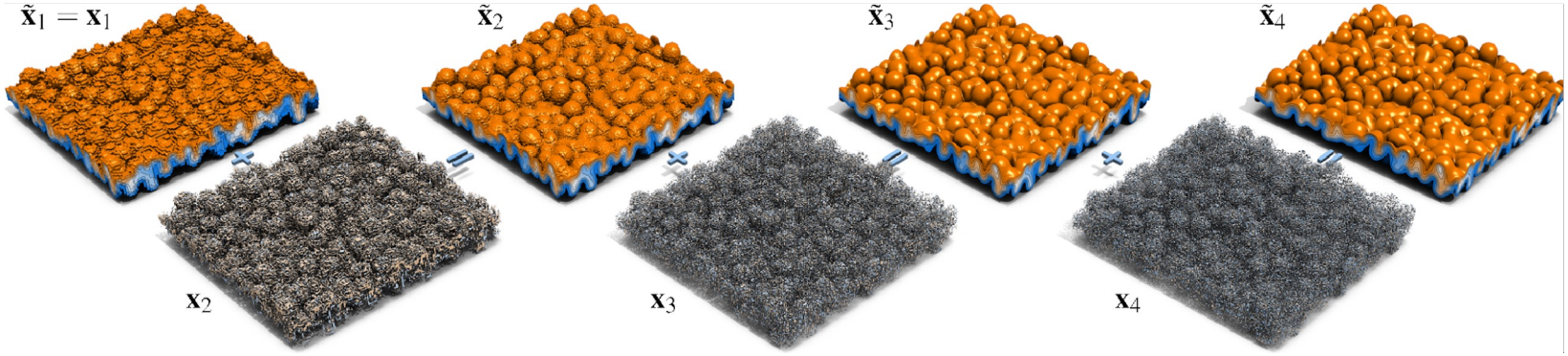
Coding: Embedded codes

- Methods covered so far encode whole values
  - I.e., we get values either at full or no precision
- In embedded coding, we encode groups of values at increasing precision
  - For example, one “bit plane” at a time
  - Enables progressive-precision coding
- Bit stream can be truncated at any point
  - For example, to fit within a given bit budget
  - Decoder reconstructs “best” approximation from available data
  - Lossless code if bit stream is not truncated



Multi-precision progressive compression [MPC] framework is agnostic wrt. compressor, file format, and API

- Idea: Express field as sum of monotonically decreasing error terms, $\{\mathbf{x}_i\}$
  - Error terms respect sequence of user-prescribed tolerances, $\tau_1 > \tau_2 > \tau_3 > \cdots$
  - i^{th} error term (a field) is compressed using *any* lossy compressor to within tolerance τ_i
    - Lossy compression error becomes $(i+1)^{\text{st}}$ error term
  - Improve approximation, $\tilde{\mathbf{x}}$, by adding to it the next error term, $\mathbf{x}_i$ (e.g., when it arrives)
  - Store **sequence of compressed fields** $\{\mathbf{x}_i\}$ whose partial sum converges to ground truth
    - For enough error terms, **achieves lossless compression** using lossy compressor!
  - Analogy: 3.14159265… = 3 + 1e–1 + 4e–2 + 2e–3 – 4e–4 – 1e–5 + 3e–6 – 3e–7 – 5e–8 + ⋯



References

[BLOSC] Alted, “BLOSC, an extremely fast, multi-threaded, meta-compressor library,” www.blosc.org

[EC] Elias, “Universal codeword sets and representations of the integers,” 1975

[EZW] Shapiro, “Embedded image coding using zerotrees of wavelet coefficients,” 1993

[FPZIP] Lindstrom & Isenburg, “Fast and efficient compression of floating-point data,” 2006

[HVQ] Schneider & Westermann, “Compression domain volume rendering,” 2003

[IC] Moffat & Stuiver, “Binary interpolative coding for effective index compression,” 2000

[ISABELA] Lakshminarasimhan et al., “ISABELA for effective in situ compression of scientific data,” 2013

[ISOBAR] Schendel et al., “ISOBAR preconditioner for effective and high-throughput lossless data compression,” 2012

[JPEG2000] Woodring et al., “Revisiting wavelet compression for large-scale climate data using JPEG 2000 and ensuring data precision,” 2011

[LP] Ibarria et al., “Out-of-core compression and decompression of large n-dimensional scalar fields,” 2003

[LZ4A/P] Kunkel et al., “Toward decoupling the selection of compression algorithms from quality constraints,” 2017

[MGARD] Ainsworth et al., “Multilevel techniques for compression and reduction of scientific data—the univariate case,” 2018

References (continued)

[MPC] Magri & Lindstrom, “A general framework for progressive data compression and retrieval,” 2023

[RBUC] Moffat & Anh, “Binary codes for non-uniform sources,” 2005

[SP] Ibarria et al., “Spectral interpolation on 3 × 3 stencils for prediction and compression,” 2007

[SPECK] Islam & Pearlman, “An embedded and efficient low-complexity hierarchical image coder,” 1999

[SPERR] Li et al., “Lossy scientific data compression with SPERR,” 2023

[SPIHT] Said & Pearlman, “A new, fast, and efficient image codec based on set partitioning in hierarchical trees,” 1996

[SQ] Iverson et al., “Fast and effective lossy compression algorithms for scientific datasets,” 2012

[SZ] Di & Cappello, “Fast error-bounded lossy HPC data compression with SZ,” 2016

[TC] Teuhola, “Tournament coding of integer sequences,” 2008

[TTHRESH] Ballester-Ripoll et al., “TTRESH: Tensor compression for multidimensional visual data,” 2019

[VAPOR] Clyne et al., “Interactive desktop analysis of high resolution simulations,” 2007

[ZFP] Lindstrom, “Fixed-rate compressed floating-point arrays,” 2014



Acknowledgment

This research was supported by the Exascale Computing Project (17-SC-20-SC), a joint project of the U.S. Department of Energy's Office of Science and National Nuclear Security Administration, responsible for delivering a capable exascale ecosystem, including software, applications, and hardware technology, to support the nation's exascale computing imperative.

Disclaimer

This document was prepared as an account of work sponsored by an agency of the United States government. Neither the United States government nor Lawrence Livermore National Security, LLC, nor any of their employees makes any warranty, expressed or implied, or assumes any legal liability or responsibility for the accuracy, completeness, or usefulness of any information, apparatus, product, or process disclosed, or represents that its use would not infringe privately owned rights. Reference herein to any specific commercial product, process, or service by trade name, trademark, manufacturer, or otherwise does not necessarily constitute or imply its endorsement, recommendation, or favoring by the United States government or Lawrence Livermore National Security, LLC. The views and opinions of authors expressed herein do not necessarily state or reflect those of the United States government or Lawrence Livermore National Security, LLC, and shall not be used for advertising or product endorsement purposes.